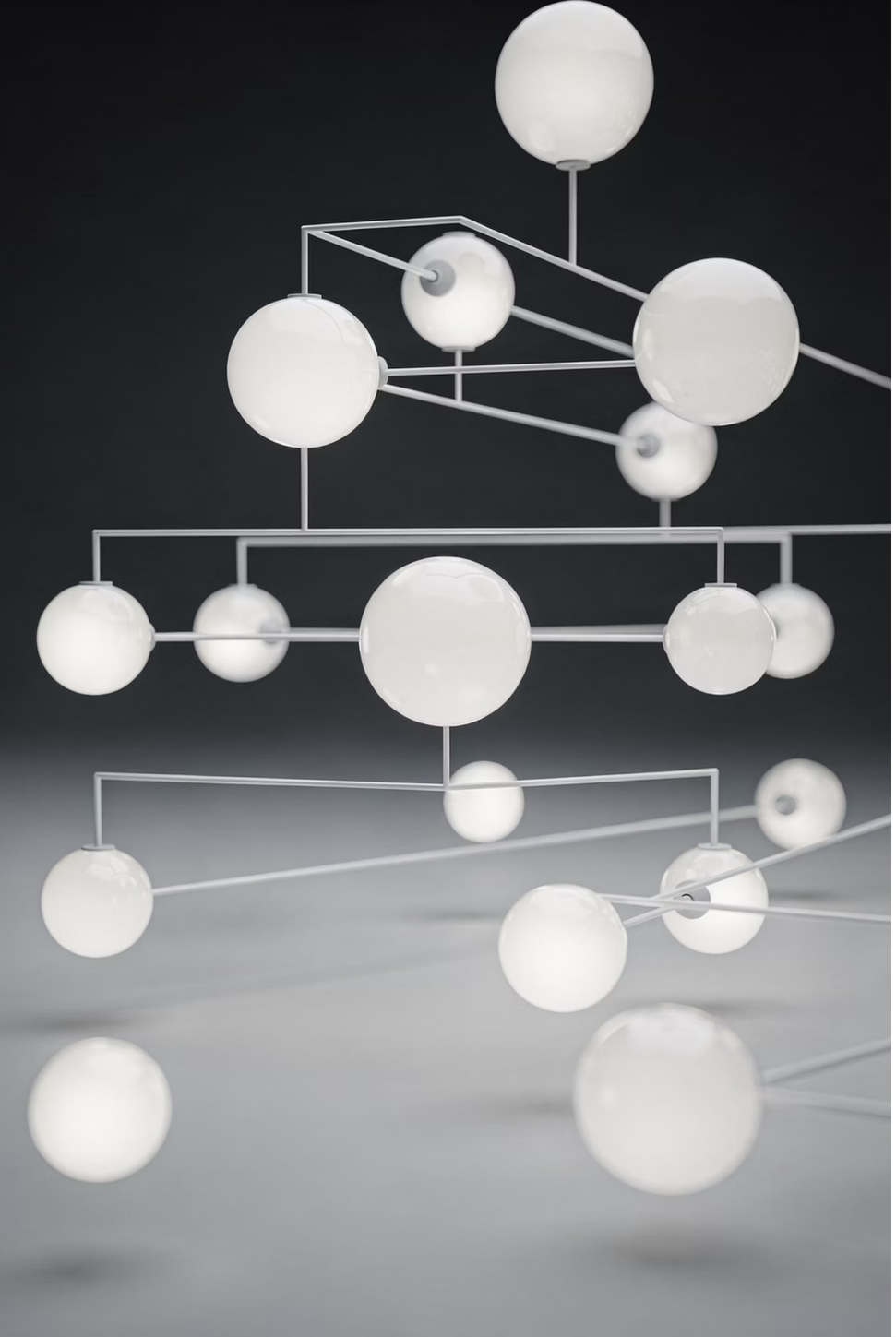


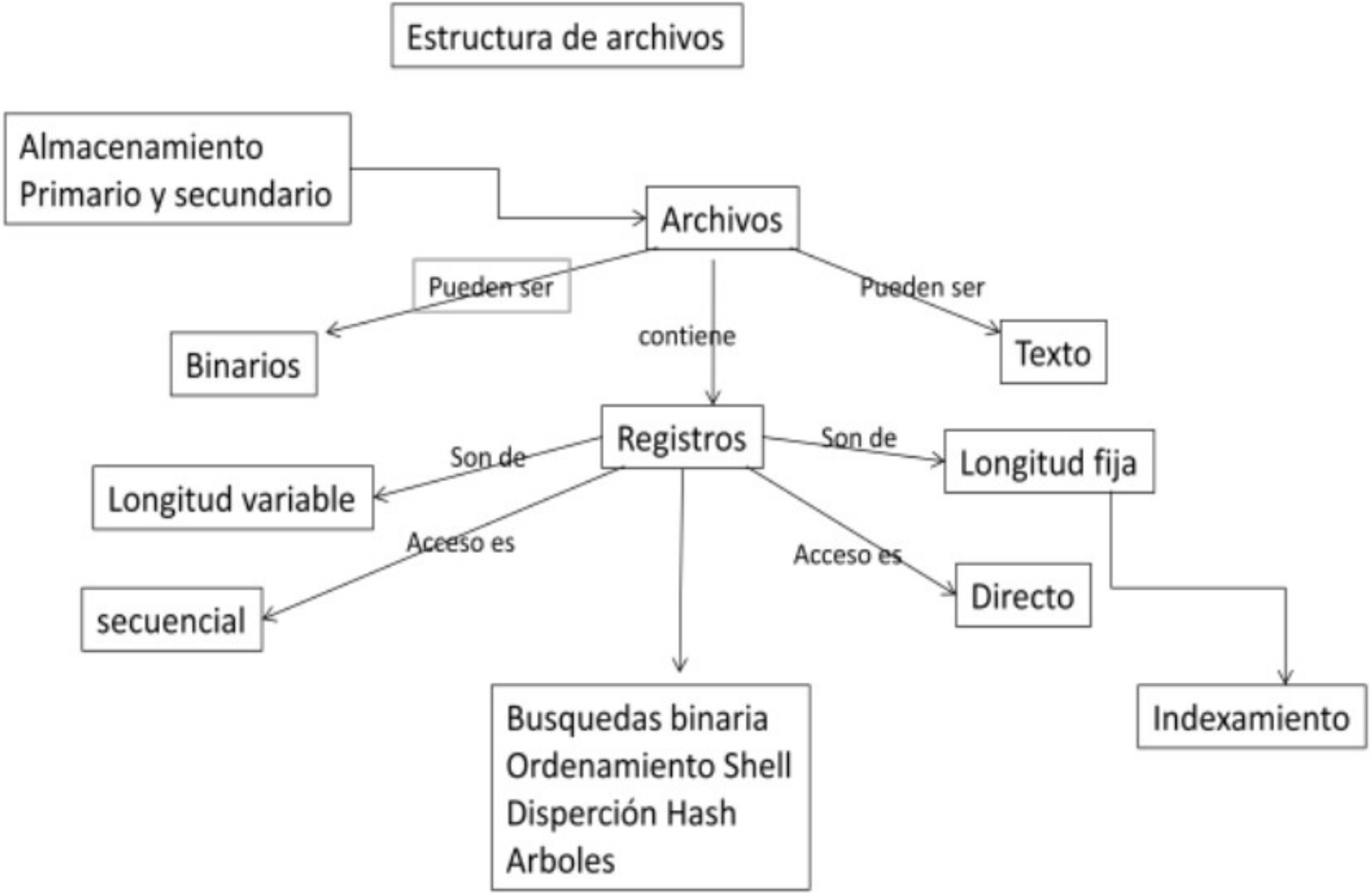
Conceptos: Estructuras de Árboles

Una exploración visual y conceptual de las estructuras de archivos, árboles binarios, montículos, árboles AVL y sus aplicaciones en almacenamiento y búsqueda eficiente.

por Carlos Alberto Rico

Valeria A. Di Croce

ESTRUCTURA CONCEPTUAL



Árboles: Repaso y Definición

Características Clave

Los árboles son estructuras **no lineales** y **dinámicas**.

Dinámica: la estructura puede variar durante la ejecución
No lineal: a cada elemento pueden seguirle varios elementos

Definición de Árbol de tipo T

Un árbol de tipo T es una estructura de datos que cumple:
Es una estructura homogénea → todos los datos almacenados son del mismo tipo T.
Se define recursivamente:

- **Un árbol es:**
  - **vacío, o**
  - **un nodo raíz que contiene un elemento de tipo T más una colección finita de subárboles disjuntos.**

Representación de Árboles

Un árbol puede representarse de cinco formas distintas:

Diagramas de Venn

Rectángulos anidados que muestran la jerarquía padre-hijo.

2 Anidación de Paréntesis

`(A((B(C))D))`

3 Notación Decimal

`1.A, 1.1.B, 1.1.1.C, 1.2.D`

Notación Indentada

Barras horizontales de longitud proporcional al nivel de profundidad.

5 Grafos

Nodos circulares conectados por aristas que representan relaciones padre-hijo.

Características y Propiedades de los Árboles

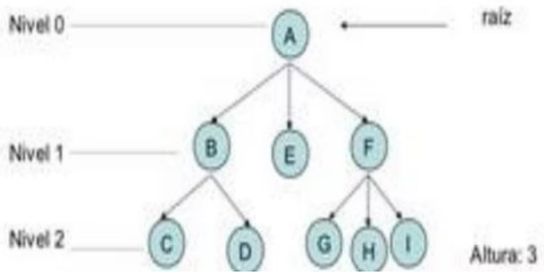
- Todo árbol no vacío tiene un único **nodo raíz**.
- Si X es descendiente directo de Y, **X es hijo de Y** e Y es padre de X.
- Todos los nodos del mismo padre son **hermanos**.
- Nodo sin hijos: **nodo terminal (hoja)**. Nodo que no es raíz ni terminal: **nodo interior**.

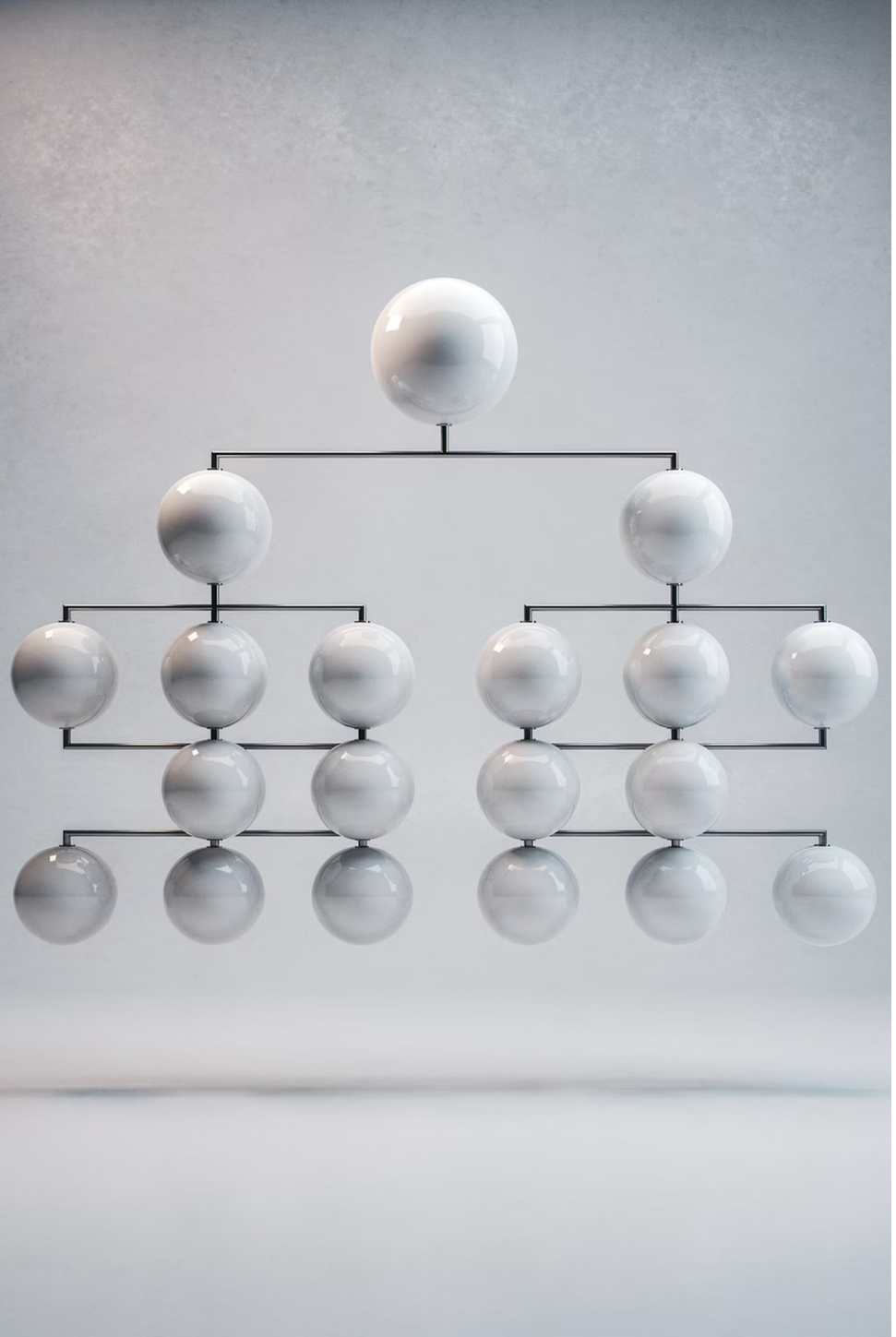
Grado

Número de descendientes directos de un nodo. El **grado del árbol** es el máximo grado de todos sus nodos.

Nivel y Altura

Nivel: nodos recorridos desde la raíz hasta un nodo. **Altura:** máximo número de niveles + 1.





Árboles Binarios

Un árbol binario es un árbol de **grado 2**: cada nodo puede tener como máximo **dos subárboles**, distinguiendo siempre entre subárbol **izquierdo** y **derecho**.

Árbol Estricto

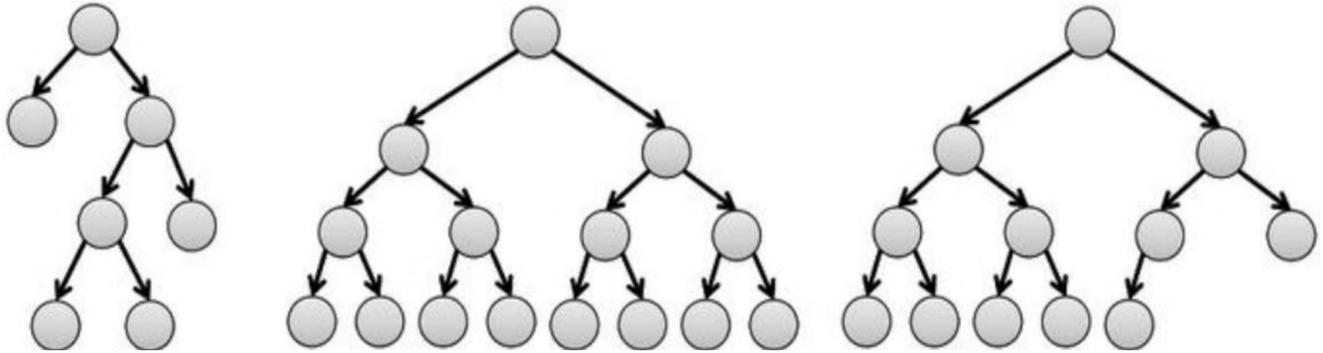
Si un subárbol está vacío, el otro también. Cada nodo tiene **0 ó 2 hijos**.

Árbol Lleno

Árbol estricto donde la altura del subárbol izquierdo es igual a la del derecho en cada nodo.

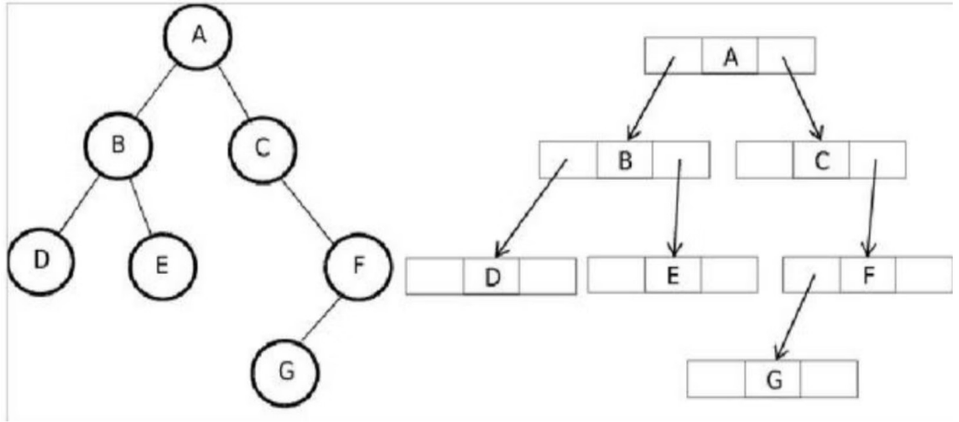
Árbol Completo

Árbol lleno hasta el penúltimo nivel. En el último nivel los nodos están agrupados **a la izquierda**.

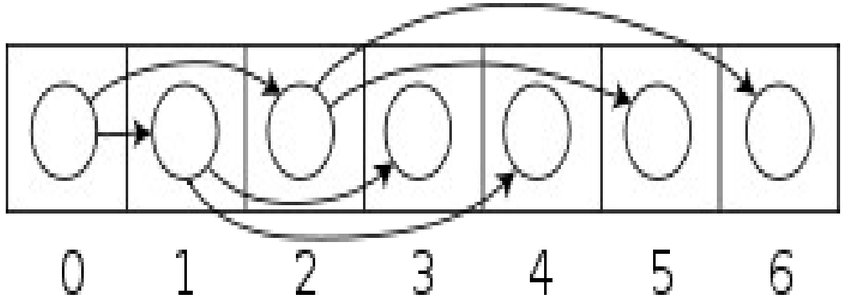


Representación de Árboles Binarios

1- Por punteros dinámicos con estructura nodo: IZQ | INFO | DER.



2- Con arreglos o listas enlazadas.



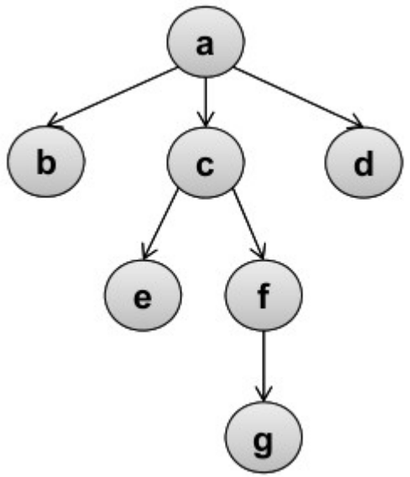
Donde el 2i+1 Hijo Izquierdo y 2i+2 hijo Derecho.
Util para Complete Binary o Full Binary.

Recorridos de Árboles Binarios

En Anchura – (a lo ancho – por niveles).

Por Niveles: Se etiquetan los nodos según su profundidad (nivel). Se recorren ordenados de menor a mayor nivel, a igualdad de nivel se recorren de izquierda a derecha.

- No recursivo: Se introduce el raiz en una cola y se entra en un bucle en el que se extrae de la cola un nodo, se recorre su elemento y se insertan sus hijos en la cola.



Por Niveles: (a) (b c d) (e f) (g

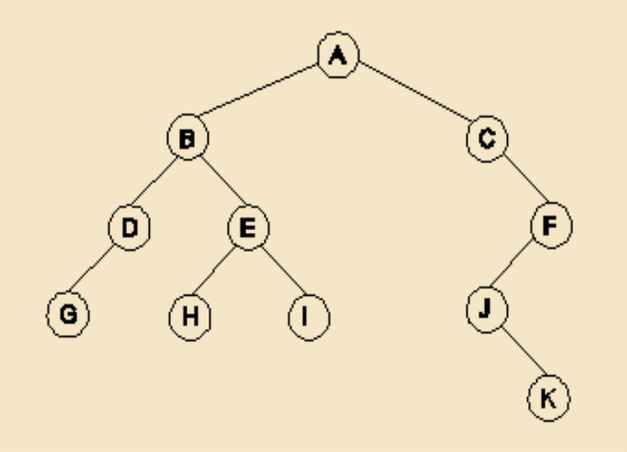
[Simulacion Árbol Binario]

En Profundidad

Recorrer significa visitar los nodos del árbol en forma sistemática, de tal manera que todos los nodos del mismo sean visitados una sola vez.
Existen distintas formas de recorrer un árbol:

1. En anchura: consiste en todos los elementos del mismo nivel, y posteriormente pasar a todos los elementos del siguiente nivel.

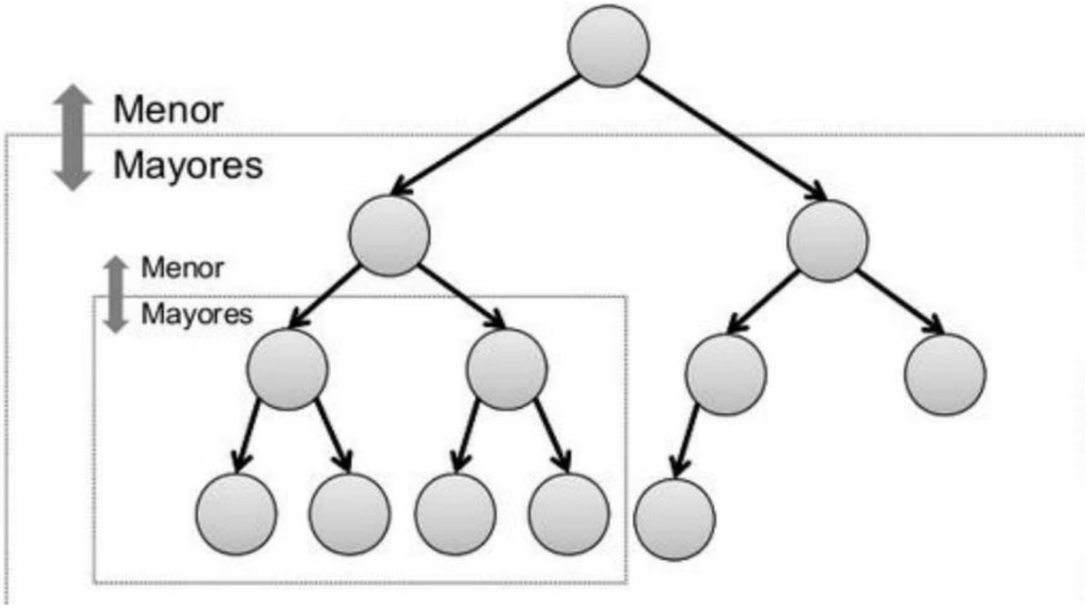
2. En profundidad:
   a. Preorden:
      - Visitar raíz
      - Recorrer subárbol izquierdo
      - Recorrer subárbol derecho
   b. Inorden:
      - Recorrer subárbol izquierdo
      - Visitar raíz
      - Recorrer subárbol derecho
   c. Postorden:
      - Recorrer subárbol izquierdo
      - Recorrer subárbol derecho
      - Visitar raíz




```
Inorden: GDBHEIACJKF
Preorden: ABDGEHICFJK
Postorden: GDHIEBKJFCA
```

Montículo Binario (Heap)

Un **montículo binario** es un árbol completo cuyos nodos almacenan elementos comparables, donde todo nodo cumple la **propiedad de montículo**: todo nodo es **menor que sus descendientes** (montículo de mínimos)



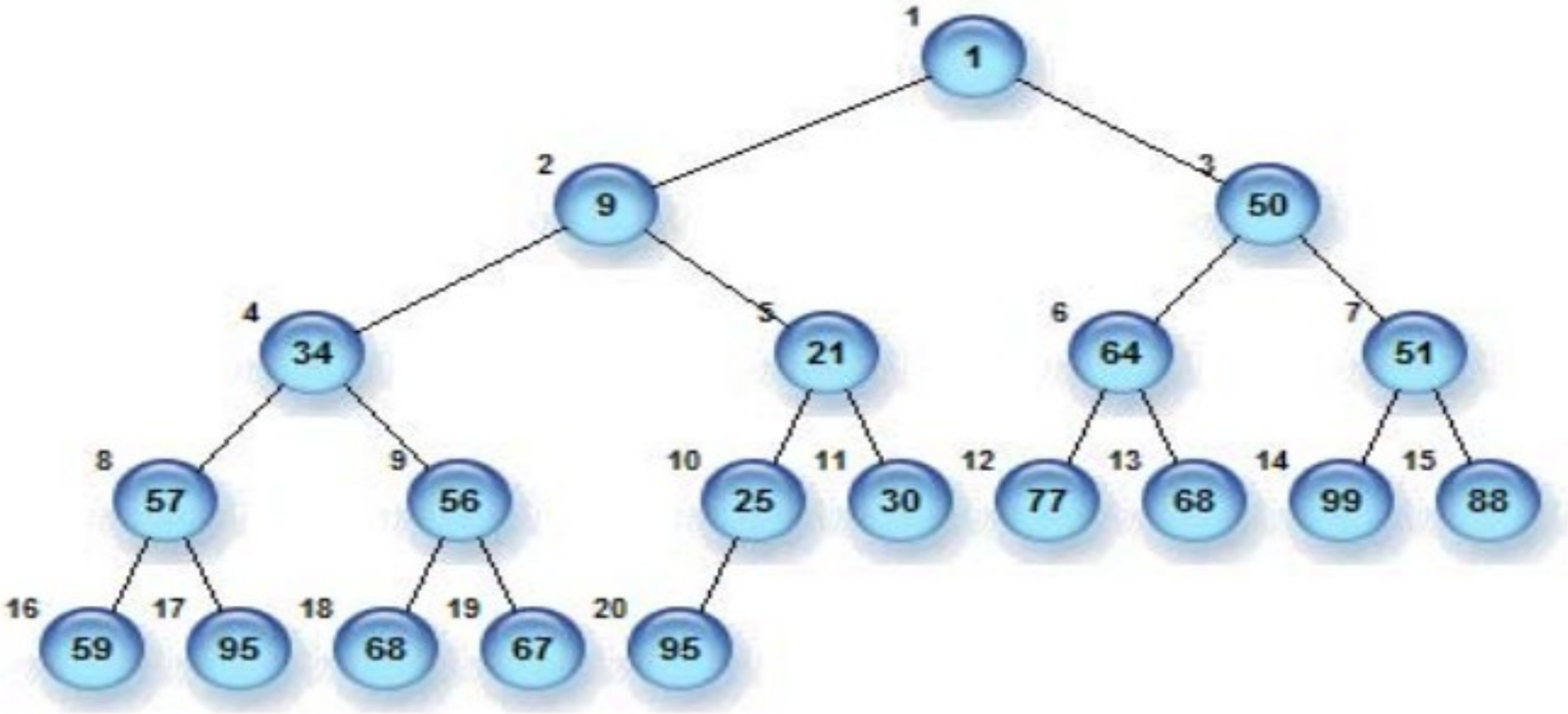
Propiedades del Montículo

El **nodo raíz** es el **mínimo**
Altura **logarítmica** respecto al número de elementos
Inserción: se sitúa al final y **asciende** hasta cumplir la propiedad
Eliminar raíz: se intercambia con el último y **desciende** la nueva raíz

Utilidad: TAD Cola de Prioridad

- Acceso al mínimo:
- Inserción y borrado del mínimo:
- Creación desde vector:

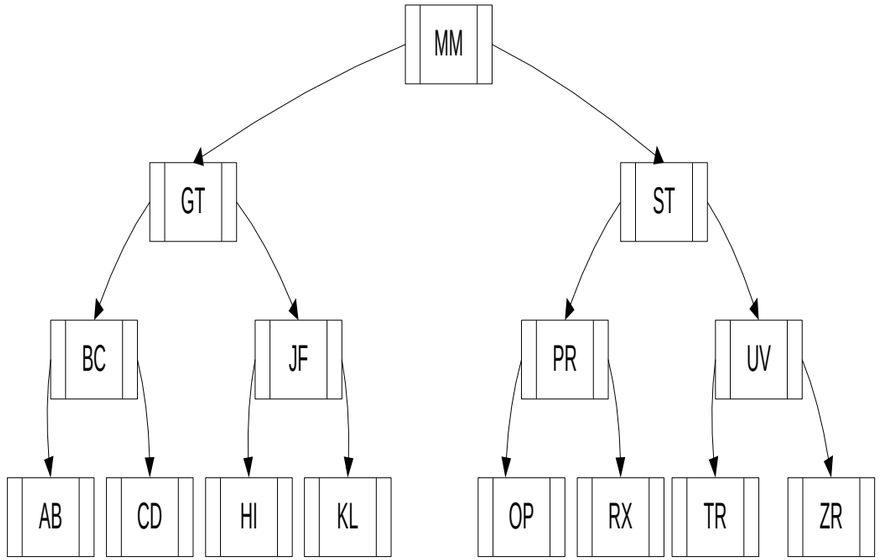
Montículo Binario



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	9	50	34	21	64	51	57	56	25	30	77	68	99	88	59	95	68	67	95

[Simulación Monticulo Binario]

Almacenamiento de un Arbol B. en disco



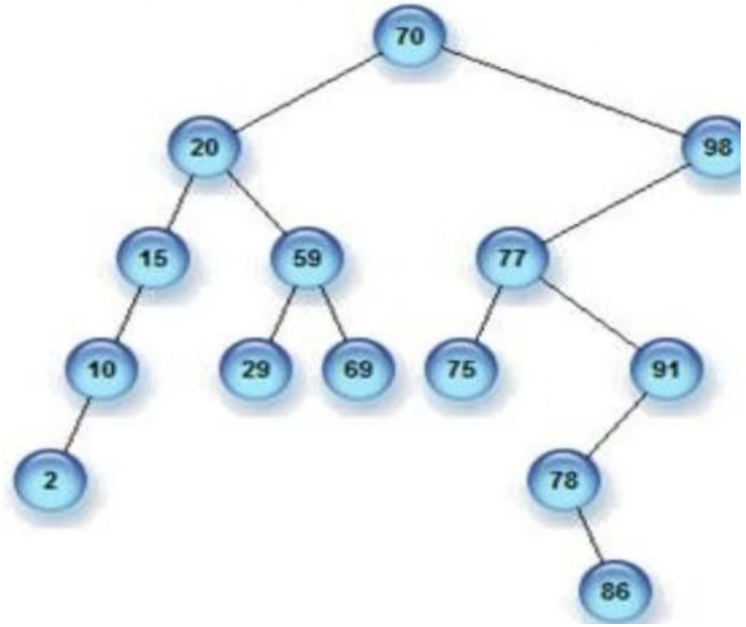
Raíz à 0

	Clave	Hijo izq	Hijo Der
0	MM	1	2
1	GT	3	4
2	ST	8	11
3	BC	5	6
4	JF	7	14
5	AB	-1	-1
6	CD	-1	-1
7	HI	-1	-1

	Clave	Hijo izq	Hijo Der
8	PR	9	10
9	OP	-1	-1
10	RX	-1	-1
11	UV	12	13
12	TR	-1	-1
13	ZR	-1	-1
14	KL	-1	-1

Árbol de Búsqueda Binario (ABB)

Un **árbol BB** es un árbol binario donde todo nodo cumple la **propiedad de ordenación**: todo nodo es **mayor** que los nodos de su subárbol izquierdo y **menor** que los de su subárbol derecho



Operaciones y Eficiencia

Recorrido **inorden** → elementos en orden ascendente

- Búsqueda, inserción y borrado:
- Acceso al i-ésimo menor:

Un árbol **equilibrado** garantiza

$$h \in O(\log n)$$

. Los árboles BB **no garantizan** equilibrio: insertar en orden produce un árbol lineal con altura

$$O(n)$$

.

Árboles Balanceados

Un árbol está **balanceado** cuando la altura de la trayectoria más corta hacia una hoja no difiere de la altura de la trayectoria más larga. El balanceo hace las búsquedas más eficientes.

Problema de los Árboles BB

Se desequilibran fácilmente. Insertar claves en orden produce un **árbol lineal** con altura , arruinando la eficiencia de todas las operaciones.

Árbol Degenerado

Cada nodo tiene un solo hijo, comportándose como una **lista**. Se pierde la eficiencia característica de los árboles.

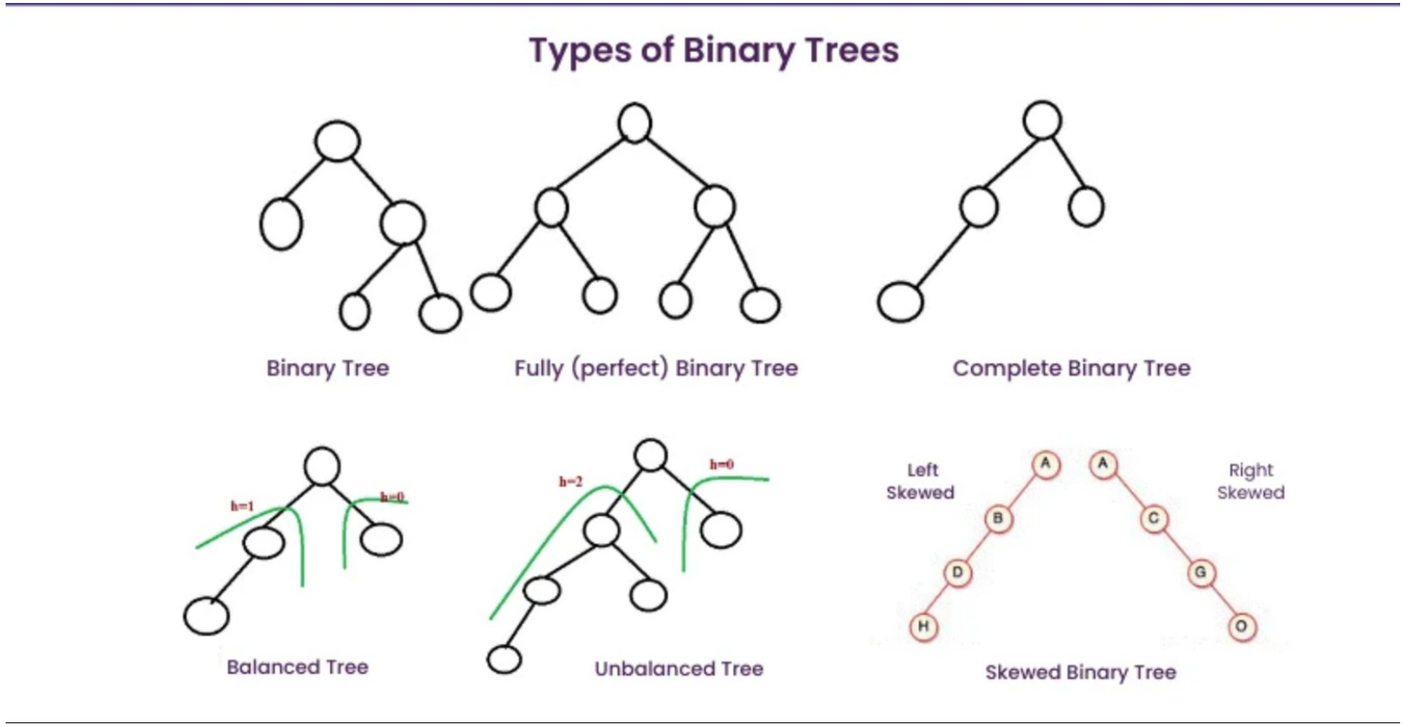
Solución: Árboles AVL

Garantizan equilibrio en todo momento mediante **rotaciones** tras cada inserción o eliminación.

Árboles AVL

Un árbol AVL es un árbol binario de búsqueda balanceado.

Un **árbol AVL** (Adelson-Velski y Landis) es un árbol binario de búsqueda que siempre está balanceado. Tras cada inserción o eliminación se comprueba el equilibrio y se realizan **rotaciones** si es necesario.



Árbol NO balanceado

```
10
  \
   20
     \
      30
        \
         40
```

Se vuelve una lista

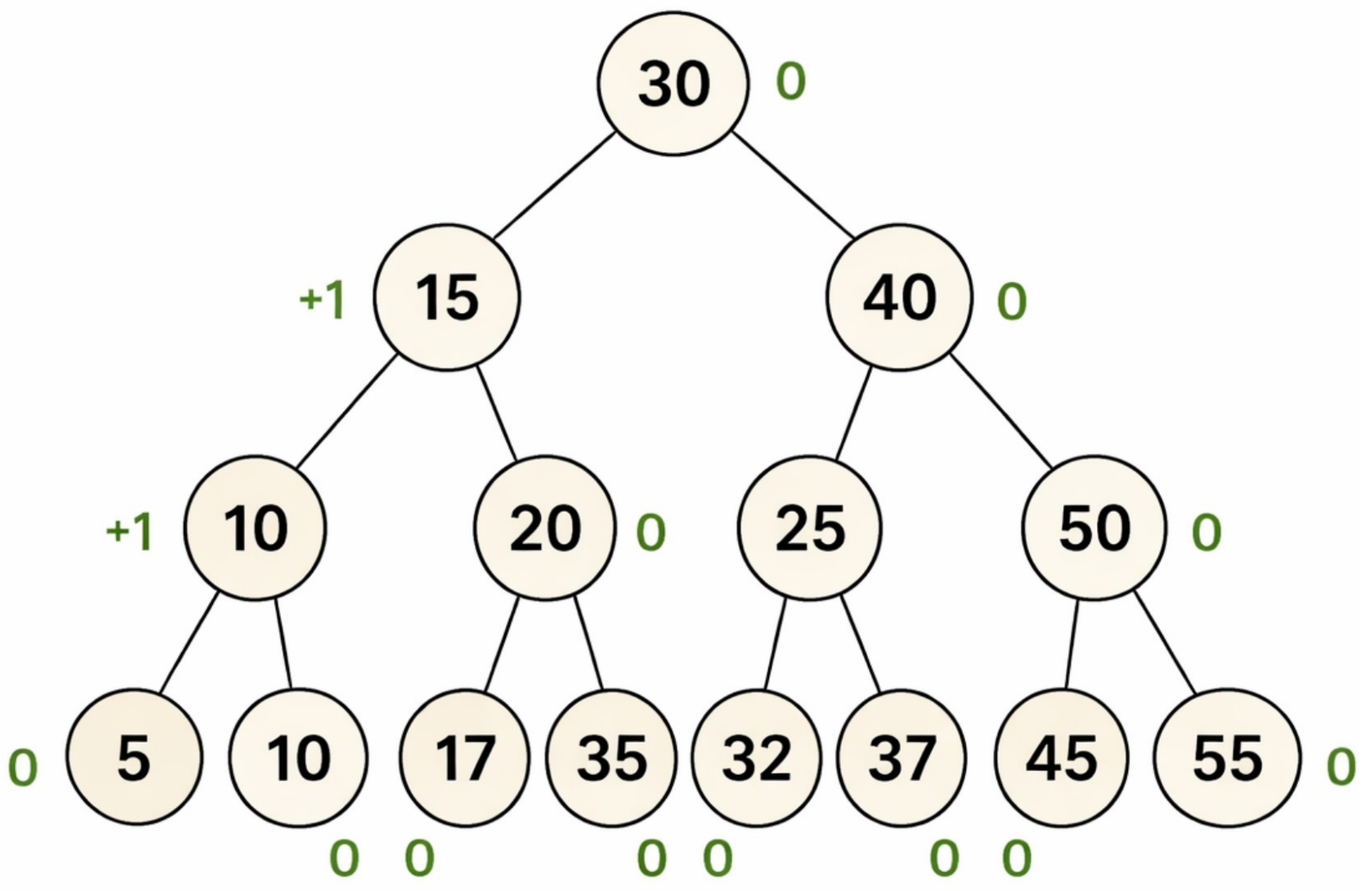
Búsqueda: O(n)

Idea fuerte de porque balancear

“Menos altura = menos pasos de búsqueda”

Árboles AVL

Un árbol AVL es un árbol binario de búsqueda balanceado.



Definición

Un **árbol AVL** es un **árbol binario de búsqueda balanceado**.

Cumple siempre:

- Para cada nodo:

  `|altura(izq) - altura(der)| ≤ 1`

Idea clave

- Mantiene el árbol “equilibrado”
- Evita que se deforme como una lista

Objetivo

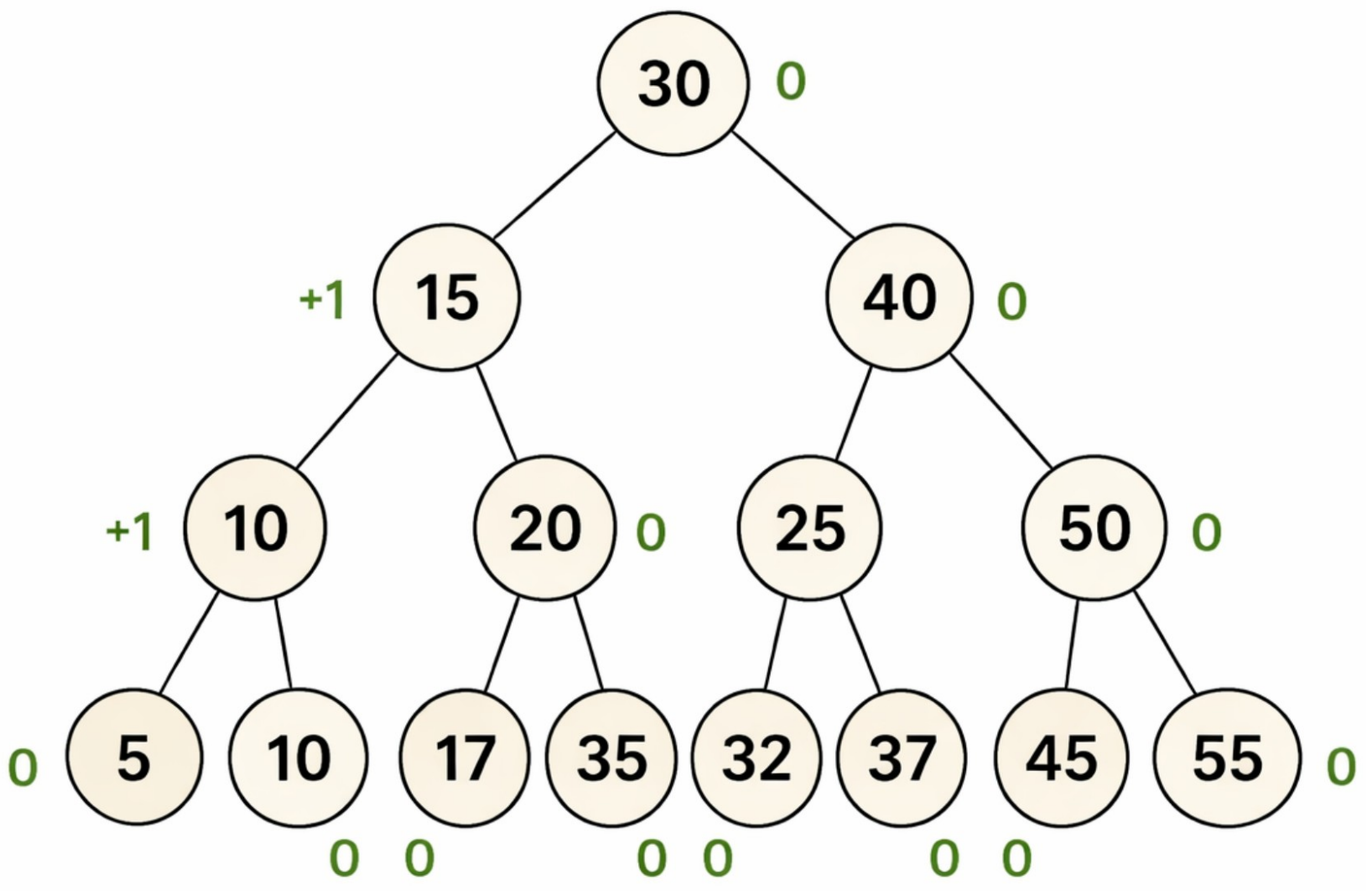
Garantizar búsquedas rápidas

Idea fuerte de porque balancear

“Menos altura = menos pasos de búsqueda”

Árboles AVL

Un árbol AVL es un árbol binario de búsqueda balanceado.



Valores posibles en AVL

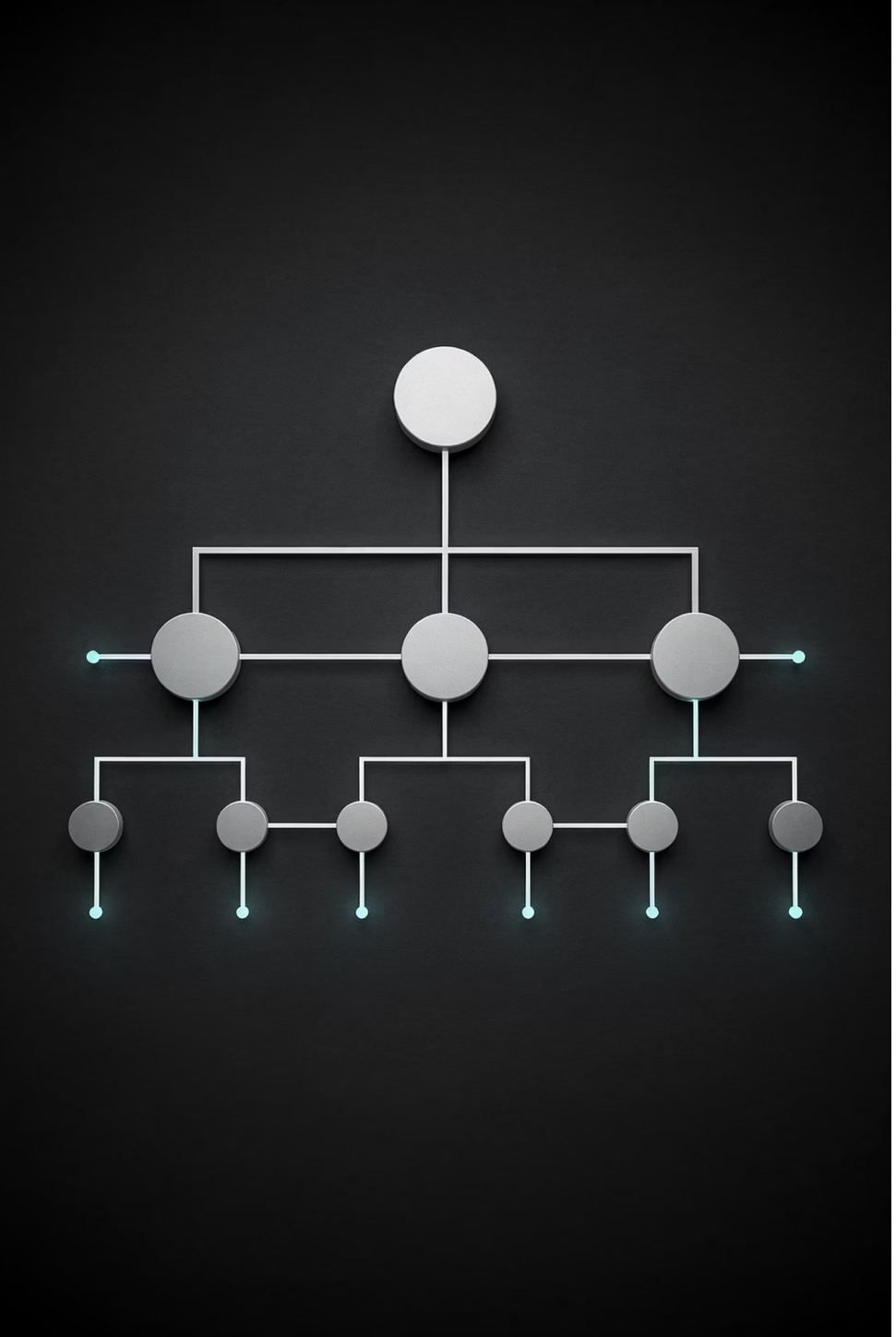
FB	Significado
0	Perfectamente balanceado
+1	Ligeramente cargado a la izquierda
-1	Ligeramente cargado a la derecha

Mediante ROTACIONES

Cuando se desbalancea:

- Rotación simple izquierda
- Rotación simple derecha
- Rotación doble

Simulación Árboles AVL



ÁRBOLES HILVANADOS

Problema de los Árboles Binarios

Recorrido costoso

Los programas recursivos implican gasto adicional de memoria y tiempo de ejecución.

Punteros NULL

Si un árbol tiene **n nodos**, existen **2n enlaces**, donde **n+1 son vacíos**.

Solución: Hilvanes

Perlis y Thorton propusieron usar los enlaces vacíos para apuntar a otros nodos del árbol, logrando recorridos más eficientes.

ÁRBOLES HILVANADOS

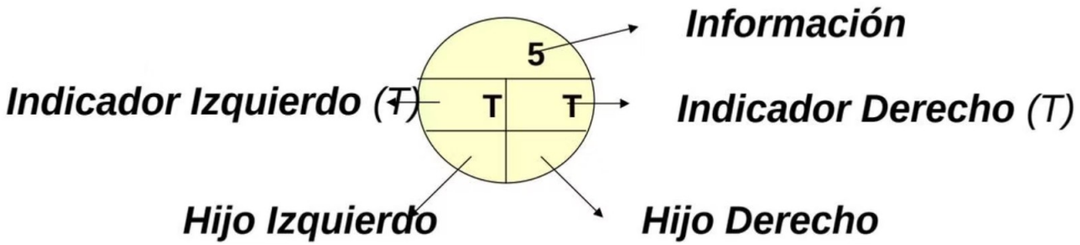
Definición y Estructura

¿Qué es un árbol hilvanado?

Un árbol binario donde cada hijo izquierdo nulo apunta al nodo **anterior en orden simétrico**, y cada hijo derecho nulo apunta al nodo **siguiente en orden simétrico**. Estos enlaces especiales se llaman **HILVANES**.

Estructura de cada nodo

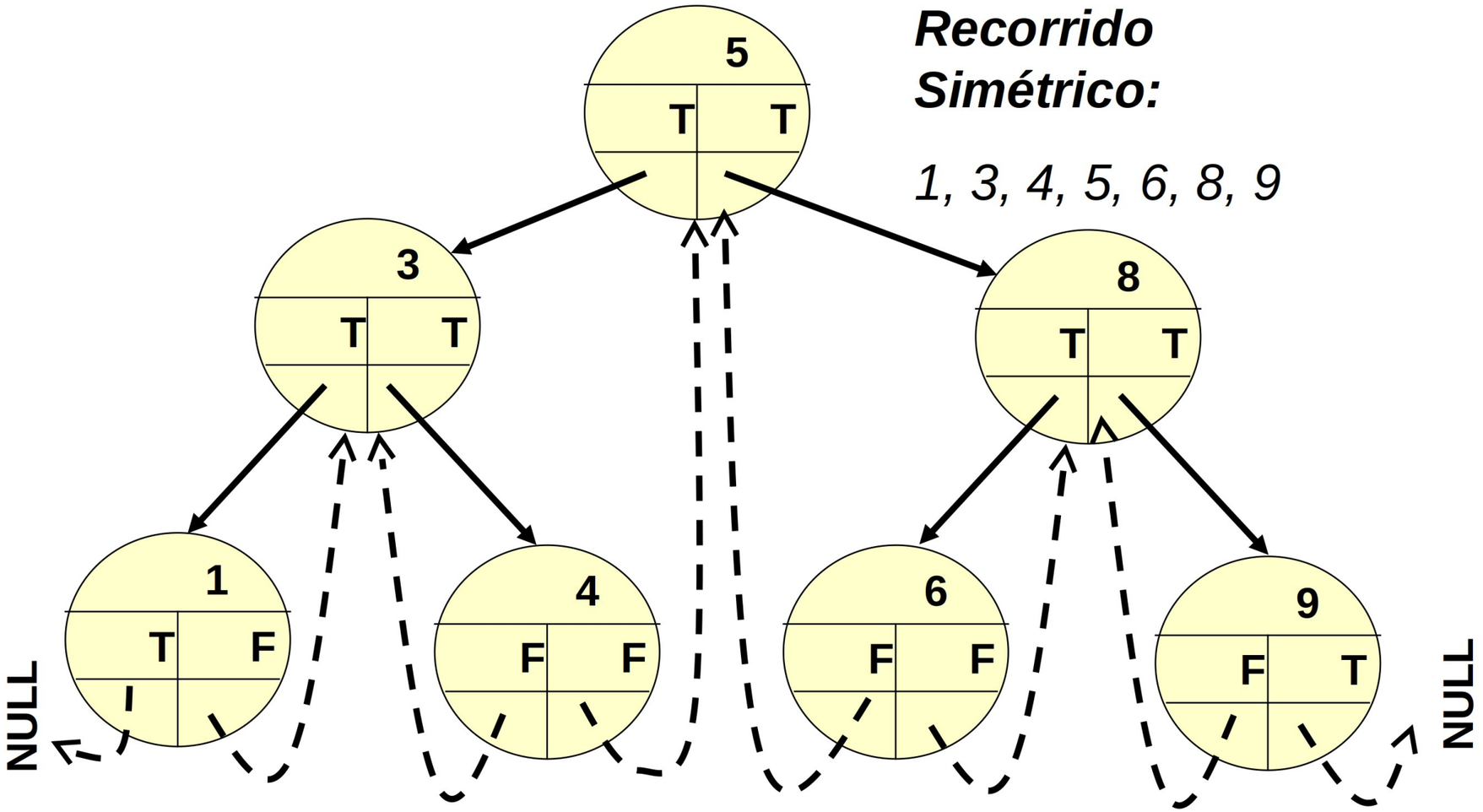
- Referencia a su información
- Apuntador al hijo izquierdo
- Indicador Izquierdo (Verdadero/Falso)
- Apuntador al hijo derecho
- Indicador Derecho (Verdadero/Falso)



ÁRBOLES HILVANADOS

Ejemplo: Árbol Hilvanado con Recorrido Simétrico

El árbol muestra el recorrido simétrico: **1, 3, 4, 5, 6, 8, 9**. Los nodos con punteros vacíos (F) usan hilvanes (flechas punteadas) para apuntar al nodo anterior o siguiente en el recorrido inorden, eliminando la necesidad de recursión.



AVL vs. BB vs Hilvanados : Características y Conclusiones

Árbol Binario (BB)

Búsqueda en el peor caso: . No garantiza equilibrio; insertar en orden produce altura .

Árbol AVL

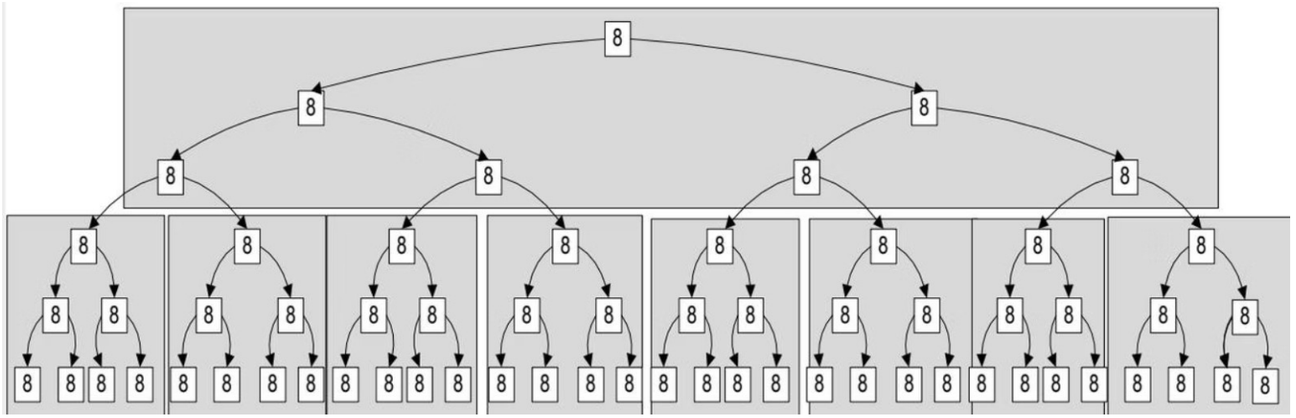
Búsqueda en el peor caso: . Siempre balanceado mediante rotaciones locales.

Árbol Hilvanado

Búsqueda en el peor caso: No mejora la altura. Mejora el recorrido por los hilos

Limitación en Disco

Ni BB ni AVL son adecuados como índices en disco: cada nivel implica un acceso a disco, y la altura logarítmica sigue siendo costosa para almacenamiento secundario.



Almacenamiento en Disco y Árboles Paginados

El almacenamiento en disco requiere **minimizar accesos**. Cada nodo visitado implica una operación de E/S costosa.

Datos útiles

Concepto	Nivel	Tamaño	Función principal
Sector	Hardware	512B / 4KB	Lectura/escritura física
Página	DBMS / SO	4KB–16KB	Transferencia memoria ↔ disco
Bloque	File System	4KB típico	Organización de datos
Cluster	File System	Variable	Asignación de espacio

Relación x Niveles

Nivel	Unidad	Rol en B-Tree
Hardware	Sector	Invisible para el diseño
Sistema de archivos	Bloque	Base física
DBMS	Página	**Unidad ideal del nodo**
Archivo	Cluster	No relevante directamente

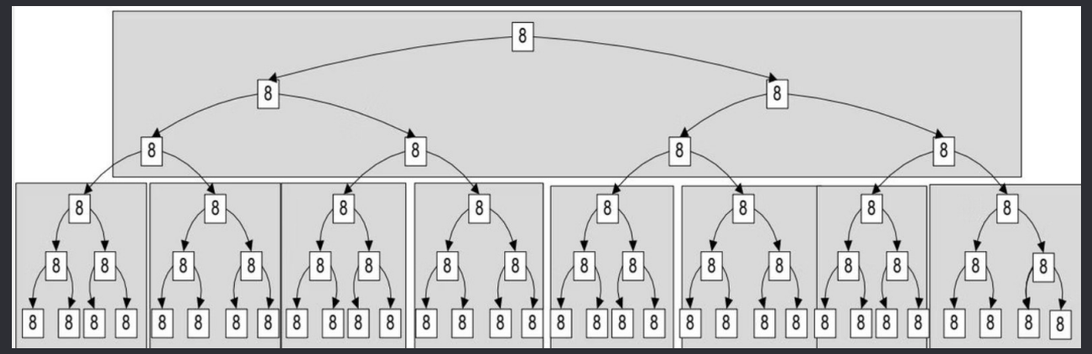
Árboles Binarios Paginados

Agrupan varios registros en una misma **página de memoria**, reduciendo el número de accesos a disco. Cada página contiene múltiples nodos.

Desafíos

- Construcción descendente: ¿cómo elegir la raíz?
- ¿Cómo mantener el árbol balanceado durante la construcción?

Gestión de **buffering** y páginas de memoria



Relación clave: nodo = página

En memoria auxiliar:

Cada nodo del árbol ≈ una página de disco

Esto se hace porque:

- Leer disco es MUY costoso
- Entonces se optimiza para:

“leer la mayor cantidad de información posible en un solo acceso”

Accesos a disco = niveles del árbol

Cada nivel del árbol implica:

1 acceso a disco (leer una página)

Entonces:

- Altura 3 → 3 accesos
- Altura 4 → 4 accesos

Esto es CRÍTICO en performance

Resumen tipo tabla

Nivel	Unidad	Quién la usa	Función
Hardware	Sector	Disco	mínima unidad física
SO	Bloque	Kernel	lectura/escritura
FS	Clúster	Sistema de archivos	asignación
Memoria	Página	MMU / SO	memoria virtual

Idea central

Cada acceso a disco es carísimo

- CPU: nanosegundos
- RAM: nanosegundos
- Disco: milisegundos

Diferencia: **millones de veces**

CONCLUSIÓN: No se usan árboles binarios, AVL ni hilvanados como índices en archivos porque generan demasiados accesos a disco debido a su baja ramificación (solo 2 hijos por nodo).

Muchos accesos a disco (I/O).